SKILL PROGRAMS

```c
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define HISTORY_CAPACITY 10

// Command history
char *history[HISTORY_CAPACITY];
int history_size = 0;

// Add command to history
void add_history(const char *cmd) {
    if (history_size < HISTORY_CAPACITY) {
        history[history_size++] = strdup(cmd);
    } else {
        free(history[0]);
        for (int i = 1; i < HISTORY_CAPACITY; i++)
            history[i-1] = history[i];

        history[HISTORY_CAPACITY-1] = strdup(cmd);
    }
}

// Display history
void show_history() {
    printf("Command History:\n");
    for (int i = 0; i < history_size; i++) {
        printf("%d: %s\n", i+1, history[i]);
    }
}

// Free history
void free_history() {
    for (int i = 0; i < history_size; i++) {
        free(history[i]);
    }
}

// Execute pipeline
void execute_pipeline(char *input) {
    char *commands[10];
    int cmd_count = 0;

    char *tok = strtok(input, "|");
    while (tok && cmd_count < 10) {
        commands[cmd_count++] = strdup(tok);
        tok = strtok(NULL, "|");
    }

    int pipes[18];
```

```
    for (int i = 0; i < cmd_count-1; i++) {
        if (pipe(pipes + i*2) < 0) {
            perror("pipe failed");
            exit(1);
        }
    }

    for (int i = 0; i < cmd_count; i++) {
        pid_t pid = fork();
        if (pid < 0) {
            perror("fork failed");
            exit(1);
        }
        if (pid == 0) {
            // Redirect input
            if (i > 0) {
                dup2(pipes[(i-1)*2], 0);
            }
            // Redirect output
            if (i < cmd_count-1) {
                dup2(pipes[i*2+1], 1);
            }
            // Close all pipes
            for (int j = 0; j < 2*(cmd_count-1); j++) {
                close(pipes[j]);
            }
            // Tokenize command
            char *args[10];
            int argc = 0;
            char *arg = strtok(commands[i], " \t");
            while (arg && argc < 9) {
                args[argc++] = arg;
                arg = strtok(NULL, " \t");
            }
            args[argc] = NULL;
            execvp(args[0], args);
            perror("exec failed");
            exit(1);
        }
    }

    // Close all pipes in parent[cite: 1]
    for (int j = 0; j < 2*(cmd_count-1); j++) {
        close(pipes[j]);
    }

    // Wait for children[cite: 1]
    for (int i = 0; i < cmd_count; i++) {
        int status;
        wait(&status);
```

```c
    for (int i = 0; i < cmd_count; i++) {
        free(commands[i]);
    }
}

int main() {
    char input[256];
    while (1) {
        printf("> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0) continue;

        add_history(input);

        if (strcmp(input, "history") == 0) {
            show_history();
            continue;
        }

        execute_pipeline(input);
    }

    free_history();
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc pipeline.c -o pipeline
shashank@LAPTOP-24KPROR8:~/OSLab$ ./pipeline
> ls -l | grep .c | sort
-rw-r--r-- 1 shashank shashank    72 Aug  4 14:00 hello.c
-rw-r--r-- 1 shashank shashank    75 Jul 31 09:43 samplePg.c
-rw-r--r-- 1 shashank shashank   104 Jul 31 10:00 Fork.c
-rw-r--r-- 1 shashank shashank   110 Jul 31 09:56 Display.c
-rw-r--r-- 1 shashank shashank   112 Aug 14 09:39 processid.c
-rw-r--r-- 1 shashank shashank   118 Aug 11 04:07 Processid.c
-rw-r--r-- 1 shashank shashank   131 Sep 18 03:10 pgm.c
-rw-r--r-- 1 shashank shashank   137 Sep 18 04:03 Memoryacc.c
-rw-r--r-- 1 shashank shashank   153 Aug 14 10:14 PC.c
-rw-r--r-- 1 shashank shashank   157 Aug 14 10:09 Exclp.c
-rw-r--r-- 1 shashank shashank   158 Jul 31 09:55 User.c
-rw-r--r-- 1 shashank shashank   159 Aug  3 13:57 Wait2.c
-rw-r--r-- 1 shashank shashank   162 Aug 14 10:04 Multichild.c
-rw-r--r-- 1 shashank shashank   167 Sep 17 16:15 escape.c
-rw-r--r-- 1 shashank shashank   225 Aug 14 09:47 processchild.c
-rw-r--r-- 1 shashank shashank   230 Aug  3 13:50 Parentchild.c
-rw-r--r-- 1 shashank shashank   237 Sep 28 15:10 Valgrind.c
-rw-r--r-- 1 shashank shashank   257 Aug 14 09:57 ParentWchild.c
-rw-r--r-- 1 shashank shashank   264 Aug  3 14:00 Exec.c
-rw-r--r-- 1 shashank shashank   264 Sep 18 03:59 Mallocfree.c
-rw-r--r-- 1 shashank shashank   274 Aug  3 13:55 Wait.c
-rw-r--r-- 1 shashank shashank   288 Sep 17 15:59 token.c
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

// Tokenize a command string into arguments
char** tokenize(char *cmd) {
    char **args = malloc(16 * sizeof(char*));
    int argc = 0;
    char *tok = strtok(cmd, " \t");
    while (tok && argc < 15) {
        args[argc++] = tok;
        tok = strtok(NULL, " \t");
    }
    args[argc] = NULL;
    return args;
}

// Execute pipeline of commands separated by |
void execute_pipeline(char *input) {
    char *commands[16];
    int cmd_count = 0;

    // Split input into commands
    char *tok = strtok(input, "|");
    while (tok && cmd_count < 16) {
        commands[cmd_count++] = strdup(tok);
        tok = strtok(NULL, "|");
    }

    int pipes[2 * (cmd_count - 1)];
    for (int i = 0; i < cmd_count - 1; i++) {
        if (pipe(pipes + i*2) < 0) {
            perror("pipe failed");
            exit(1);
        }
    }

    for (int i = 0; i < cmd_count; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            // Redirect input
            if (i > 0) {
                dup2(pipes[(i-1)*2], 0);
            }
            // Redirect output
            if (i < cmd_count - 1) {
                dup2(pipes[i*2+1], 1);
            }
```

```c
            // Close all pipes
            for (int j = 0; j < 2 * (cmd_count-1); j++) {
                close(pipes[j]);
            }

            char **args = tokenize(commands[i]);
            execvp(args[0], args);
            perror("exec failed");
            exit(1);
        }
    }

    // Close all pipes in parent
    for (int j = 0; j < 2 * (cmd_count-1); j++) {
        close(pipes[j]);
    }

    // Wait for all children
    for (int i = 0; i < cmd_count; i++) {
        int status;
        wait(&status);
    }

    for (int i = 0; i < cmd_count; i++) {
        free(commands[i]);
    }
}

int main() {
    char input[256];
    while (1) {
        printf(">");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = 0;
        if (strlen(input) == 0) continue;
        execute_pipeline(input);
    }
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano Dataflow.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc Dataflow.c -o Dataflow
shashank@LAPTOP-24KPROR8:~/OSLab$ ./Dataflow
>ls -l | grep .c | sort | wc -l
67
>echo hello world | tr a-z A-Z
HELLO WORLD
>
```

11

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

// Tokenize command into arguments
char** tokenize(char *cmd) {
    char **args = malloc(16 * sizeof(char*));
    int argc = 0;
    char *tok = strtok(cmd, " \t");
    while (tok && argc < 15) {
        args[argc++] = tok;
        tok = strtok(NULL, " \t");
    }
    args[argc] = NULL;
    return args;
}

void execute_command(char *input) {
    char *infile = NULL, *outfile = NULL;
    int append = 0;

    // Parse redirection
    char *cmd = strdup(input);
    char *tok = strtok(cmd, " \t");
    char command[256] = "";
    while (tok) {
        if (strcmp(tok, "<") == 0) {
            tok = strtok(NULL, " \t");
            infile = tok;
        } else if (strcmp(tok, ">") == 0) {
            tok = strtok(NULL, " \t");
            outfile = tok;
            append = 0;
        } else if (strcmp(tok, ">>") == 0) {
            tok = strtok(NULL, " \t");
            outfile = tok;
            append = 1;
        } else {
            strcat(command, tok);
            strcat(command, " ");
            tok = strtok(NULL, " \t");
        }
    }

    char **args = tokenize(command);
    if (!args[0]) {
        free(args);
        free(cmd);
        return;
```

```c
    if (pid == 0) {
        // Input redirection
        if (infile) {
            int fd = open(infile, O_RDONLY);
            if (fd < 0) {
                perror("Cannot open input file");
                exit(1);
            }
            dup2(fd, STDIN_FILENO);
            close(fd);
        }

        // Output redirection
        if (outfile) {
            int fd;
            if (append)
                fd = open(outfile, O_WRONLY | O_CREAT | O_APPEND, 0644);
            else
                fd = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
            if (fd < 0) {
                perror("Cannot open output file");
                exit(1);
            }
            dup2(fd, STDOUT_FILENO);
            close(fd);
        }

        execvp(args[0], args);
        perror("exec failed");
        exit(1);
    } else {
        int status;
        waitpid(pid, &status, 0);
        if (WIFEXITED(status)) {
            printf("Process exited with %d\n", WEXITSTATUS(status));
        } else {
            perror("fork failed");
        }
    }

    free(args);
    free(cmd);
}

int main() {
    char input[256];
    while (1) {
        printf(">");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = 0;
        if (strlen(input) == 0) continue;
        execute_command(input);
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano FileInputs.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc FileInputs.c -o FileInputs
shashank@LAPTOP-24KPROR8:~/OSLab$ ./FileInputs
> cat < input.txt
Cannot open input file: No such file or directory
Process exited with 1
>echo hello > out.txt
Process exited with 0
>
```

12

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

// Tokenize command into arguments
char** tokenize(char *cmd) {
    char **args = malloc(16 * sizeof(char*));
    int argc = 0;
    char *tok = strtok(cmd, " \t");
    while (tok && argc < 15) {
        args[argc++] = tok;
        tok = strtok(NULL, " \t");
    }
    args[argc] = NULL;
    return args;
}

void execute_command(char *input) {
    char *outfile = NULL, *errfile = NULL;
    int append_out = 0, append_err = 0;

    // Parse redirection
    char *cmd = strdup(input);
    char *tok = strtok(cmd, " \t");
    char command[256] = "";
    while (tok) {
        if (strcmp(tok, ">") == 0) {
            tok = strtok(NULL, " \t");
            outfile = tok;
            append_out = 0;
        } else if (strcmp(tok, ">>") == 0) {
            tok = strtok(NULL, " \t");
            outfile = tok;
            append_out = 1;
        } else if (strcmp(tok, "2>") == 0) {
            tok = strtok(NULL, " \t");
            errfile = tok;
            append_err = 0;
        } else if (strcmp(tok, "2>>") == 0) {
            tok = strtok(NULL, " \t");
            errfile = tok;
            append_err = 1;
        } else {
            strcat(command, tok);
            strcat(command, " ");
            tok = strtok(NULL, " \t");
        }
    }
```

```c
    char **args = tokenize(command);
    if (!args[0]) {
        free(args);
        free(cmd);
        return;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork failed");
        free(args);
        free(cmd);
        return;
    }

    if (pid == 0) {
        // Output redirection
        if (outfile) {
            int fd;
            if (append_out)
                fd = open(outfile, O_WRONLY | O_CREAT | O_APPEND, 0644);
            else
                fd = open(outfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
            if (fd < 0) {
                perror("Cannot open output file");
                exit(1);
            }
            dup2(fd, STDOUT_FILENO);
            close(fd);
        }

        // Error redirection
        if (errfile) {
            int fd;
            if (append_err)
                fd = open(errfile, O_WRONLY | O_CREAT | O_APPEND, 0644);
            else
                fd = open(errfile, O_WRONLY | O_CREAT | O_TRUNC, 0644);
            if (fd < 0) {
                perror("Cannot open error file");
                exit(1);
            }
            dup2(fd, STDERR_FILENO);
            close(fd);
        }

        execvp(args[0], args);
        perror("exec failed");
        exit(1);
    } else {
        int status;
        waitpid(pid, &status, 0);
```

```c
    } else {
        int status;
        waitpid(pid, &status, 0);
        if (WIFEXITED(status)) {
            printf("Process exited with %d\n", WEXITSTATUS(status));
        } else {
            perror("fork failed");
        }
    }

    free(args);
    free(cmd);
}

int main() {
    char input[256];
    while (1) {
        printf(">");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = 0;
        if (strlen(input) == 0) continue;
        execute_command(input);
    }
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano Parse.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc Parse.c -o Parse
shashank@LAPTOP-24KPROR8:~/OSLab$ ./Parse
>echo hello >> out.txt
Process exited with 0
>ls nofile 2>> errors.txt
errors.txt
Process exited with 2
>
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

// Tokenize command into arguments
char** tokenize(char *cmd) {
    char **args = malloc(32 * sizeof(char*));
    int argc = 0;
    char *tok = strtok(cmd, " \t");
    while (tok && argc < 31) {
        args[argc++] = tok;
        tok = strtok(NULL, " \t");
    }
    args[argc] = NULL;
    return args;
}

// Execute pipeline with redirection
void execute_pipeline(char *input) {
    char *commands[16];
    int cmd_count = 0;

    // Split input into pipeline segments
    char *tok = strtok(input, "|");
    while (tok && cmd_count < 16) {
        commands[cmd_count++] = strdup(tok);
        tok = strtok(NULL, "|");
    }

    int pipes[2 * (cmd_count - 1)];
    for (int i = 0; i < cmd_count - 1; i++) {
        if (pipe(pipes + i*2) < 0) {
            perror("pipe failed");
            exit(1);
        }
    }

    for (int i = 0; i < cmd_count; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            // Handle pipe redirection
            if (i > 0) dup2(pipes[(i-1)*2], STDIN_FILENO);
            if (i < cmd_count - 1) dup2(pipes[i*2+1], STDOUT_FILENO);

            // Close all pipes
            for (int j = 0; j < 2 * (cmd_count-1); j++) close(pipes[j]);

            // Parse redirection inside command
            char *outfile = NULL, *errfile = NULL;
```

```
        char *tok2 = strtok(commands[i], " \t");
        while (tok2) {
            if (strcmp(tok2, ">") == 0) {
                tok2 = strtok(NULL, " \t");
                outfile = tok2;
                append_out = 0;
            } else if (strcmp(tok2, ">>") == 0) {
                tok2 = strtok(NULL, " \t");
                outfile = tok2;
                append_out = 1;
            } else if (strcmp(tok2, "2>") == 0) {
                tok2 = strtok(NULL, " \t");
                errfile = tok2;
                append_err = 0;
            } else if (strcmp(tok2, "2>>") == 0) {
                tok2 = strtok(NULL, " \t");
                errfile = tok2;
                append_err = 1;
            } else if (strcmp(tok2, "2>&1") == 0) {
                merge_err = 1;
            } else {
                strcat(command, tok2);
                strcat(command, " ");
            }
            tok2 = strtok(NULL, " \t");
        }

        char **args = tokenize(command);
        if (!args[0]) {
            exit(0);
        }

        // Output redirection
        if (outfile) {
            int fd = open(outfile, O_WRONLY | O_CREAT | (append_out ? O_APPEND : O_TRU>
            if (fd < 0) { perror("Cannot open output file"); exit(1); }
            dup2(fd, STDOUT_FILENO);
            close(fd);
        }

        // Error redirection
        if (errfile) {
            int fd = open(errfile, O_WRONLY | O_CREAT | (append_err ? O_APPEND : O_TRU>
            if (fd < 0) { perror("Cannot open error file"); exit(1); }
            dup2(fd, STDERR_FILENO);
            close(fd);
        }

        // Merge stderr into stdout
        if (merge_err) {
            dup2(STDOUT_FILENO, STDERR_FILENO);
```

```
            // Merge stderr into stdout
            if (merge_err) {
                dup2(STDOUT_FILENO, STDERR_FILENO);
            }

            execvp(args[0], args);
            perror("exec failed");
            exit(1);
        }
    }

    // Parent closes pipes
    for (int j = 0; j < 2 * (cmd_count-1); j++) close(pipes[j]);

    // Wait for all children
    for (int i = 0; i < cmd_count; i++) {
        int status;
        wait(&status);
    }

    for (int i = 0; i < cmd_count; i++) {
        free(commands[i]);
    }
}

int main() {
    char input[256];
    while (1) {
        printf(">");
        if (!fgets(input, sizeof(input), stdin)) break;
        input[strcspn(input, "\n")] = 0;
        if (strlen(input) == 0) continue;
        execute_pipeline(input);
    }
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano Parsecomplex.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc Parsecomplex.c -o Parsecomplex
shashank@LAPTOP-24KPROR8:~/OSLab$ ./Parsecomplex
>ls -l | grep .c > out.txt 2>&1
>cat missing.txt 2> errors.txt
>echo hello >> log.txt | wc -c
0
>
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX 10

struct Job {
    int id;
    pid_t pid;
    char command[50];
    char status[15];
};

struct Job jobs[MAX];
int count = 0;

void showJobs() {
    int status;
    for (int i = 0; i < count; i++) {
        if (strcmp(jobs[i].status, "Running") == 0) {
            if (waitpid(jobs[i].pid, &status, WNOHANG) > 0) {
                strcpy(jobs[i].status, "Completed");
            }
        }
        printf("[%d] PID: %d %s %s\n", jobs[i].id, jobs[i].pid, jobs[i].command, jobs[i].s>
    }
}

int main() {
    char command[50];
    printf("Background Job Manager\n");
    while (1) {
        printf("\n>");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = 0;

        if (strcmp(command, "exit") == 0) break;

        if (strcmp(command, "jobs") == 0) {
            showJobs();
            continue;
        }

        if (command[strlen(command) - 1] == '&') {
            command[strlen(command) - 1] = 0;
            pid_t pid = fork();
            if (pid == 0) {
                execlp(command, command, NULL);
                exit(1);
            }
```

```c
        if (command[strlen(command) - 1] == '&') {
            command[strlen(command) - 1] = 0;
            pid_t pid = fork();
            if (pid == 0) {
                execlp(command, command, NULL);
                exit(1);
            }
            jobs[count].id = count + 1;
            jobs[count].pid = pid;
            strcpy(jobs[count].command, command);
            strcpy(jobs[count].status, "Running");
            count++;
            printf("[%d] Started in background\n", count);
        }
    }
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano Detection.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc Detection.c -o Detection
shashank@LAPTOP-24KPROR8:~/OSLab$ ./Detection
Background Job Manager

>sleep 5 &
[1] Started in background

>jobs
[1] PID: 1057 sleep 5  Completed

>exit
```

15

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

struct Job {
    int id;
    pid_t pid;
    int running;
};

struct Job jobs[5];
int count = 0;

void listJobs() {
    printf("\nActive Jobs:\n");
    for (int i = 0; i < count; i++) {
        printf("[%d] PID=%d Status=%s\n",
            jobs[i].id,
            jobs[i].pid,
            jobs[i].running ? "Running" : "Completed");
    }
}

void foreground(int id) {
    for (int i = 0; i < count; i++) {
        if (jobs[i].id == id) {
            printf("Bringing Job [%d] to foreground...\n", id);
            /* Resume process */
            kill(jobs[i].pid, SIGCONT);
            /* Wait for completion */
            waitpid(jobs[i].pid, NULL, 0);
            /* Update state */
            jobs[i].running = 0;
            printf("Job [%d] completed.\n", id);
            return;
        }
    }
    printf("Job not found.\n");
}

int main() {
    int choice, id;
    pid_t pid;
    printf("Job Control System\n");
    while (1) {
        printf("\n1. Start Job");
        printf("\n2. List Jobs");
        printf("\n3. Foreground Job");
        printf("\n4. Exit");
        printf("\nEnter choice: ");
```

```
        scanf("%d", &choice);

        if (choice == 1) {
            pid = fork();
            if (pid == 0) {
                execlp("sleep", "sleep", "10", NULL);
                exit(1);
            }
            jobs[count].id = count + 1;
            jobs[count].pid = pid;
            jobs[count].running = 1;
            count++;
            printf("Job [%d] started.\n", count);
        }
        else if (choice == 2) {
            listJobs();
        }
        else if (choice == 3) {
            printf("Enter Job ID: ");
            scanf("%d", &id);
            foreground(id);
        }
        else if (choice == 4) {
            break;
        }
        else {
            printf("Invalid choice.\n");
        }
    }
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano Job.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc Job.c -o Job
shashank@LAPTOP-24KPROR8:~/OSLab$ ./Job
Job Control System

1. Start Job
2. List Jobs
3. Foreground Job
4. Exit
Enter choice: 1
Job [1] started.

1. Start Job
2. List Jobs
3. Foreground Job
4. Exit
Enter choice: 8
Invalid choice.

1. Start Job
2. List Jobs
3. Foreground Job
4. Exit
Enter choice: 4
```

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

struct Job {
    int id;
    pid_t pid;
    int status; // 1 = Running, 0 = Stopped
};

struct Job job;

void checkStatus() {
    int status;
    pid_t r = waitpid(job.pid, &status, WNOHANG | WUNTRACED);
    if (r > 0) {
        if (WIFSTOPPED(status)) {
            job.status = 0;
            printf("Job [%d] Stopped\n", job.id);
        }
        else if (WIFCONTINUED(status)) {
            job.status = 1;
            printf("Job [%d] Running\n", job.id);
        }
    }
}

int main() {
    pid_t pid;
    printf("Signal and Job Recovery System\n");
    pid = fork();
    if (pid == 0) {
        /* Create a separate process group */
        setpgid(0, 0);
        while (1) {
            printf("Child process running...\n");
            sleep(2);
        }
    }

    job.id = 1;
    job.pid = pid;
    job.status = 1;
    sleep(2);

    /* Stop the job */
    printf("\nStopping Job [1]...\n");
    kill(pid, SIGSTOP);
    sleep(1);
    checkStatus();
```

```c
    /* Resume the stopped job */
    printf("\nSending SIGCONT...\n");
    kill(pid, SIGCONT);
    sleep(1);

    /* Check updated status */
    checkStatus();
    printf("\nJob resumed successfully.\n");

    /* Stop and terminate child */
    kill(pid, SIGTERM);
    waitpid(pid, NULL, 0);
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano sigcount.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc sigcount.c -o sigcount
shashank@LAPTOP-24KPROR8:~/OSLab$ ./sigcount
Signal and Job Recovery System
Child process running...

Stopping Job [1]...
Job [1] Stopped

Sending SIGCONT...
Child process running...

Job resumed successfully.
```

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child = -1;
int job_running = 0;

/* SIGINT handler */
void handle_sigint(int sig)
{
    if (child > 0 && job_running) {
        printf("\nSIGINT received. Terminating foreground job...\n");
        /* Forward SIGINT to child */
        kill(child, SIGINT);
    }
    else {
        printf("\nShell protected from SIGINT.\n");
    }
}

int main()
{
    signal(SIGINT, handle_sigint);
    printf("Simple Signal Handling Shell\n");
    while (1) {
        printf("\nEnter 1 to start job, 2 to exit: ");
        int choice;
        scanf("%d", &choice);

        if (choice == 2)
            break;

        if (choice == 1) {
            child = fork();
            if (child == 0) {
                printf("Foreground job running...\n");
                /* Simulate long-running job */
                while (1)
                    sleep(1);
            }
            job_running = 1;
            /* Shell waits while protecting itself from SIGINT */
            waitpid(child, NULL, 0);
            job_running = 0;
            child = -1;
            printf("Foreground job terminated.\n");
        }
    }
    printf("Shell exiting...\n");
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano sigint.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc sigint.c -o sigint
shashank@LAPTOP-24KPROR8:~/OSLab$ ./sigint
Simple Signal Handling Shell

Enter 1 to start job, 2 to exit: 1
Foreground job running...
```

```c
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t child;
int job_status = 1; // 1 = Running, 0 = Stopped

void handle_tstp(int sig)
{
    printf("\nSIGTSTP received. Suspending job...\n");
    kill(child, SIGSTOP);
    job_status = 0;
}

int main()
{
    int status;
    signal(SIGTSTP, handle_tstp);
    child = fork();
    if (child == 0) {
        /* Create new process group */
        setpgid(0, 0);
        while (1) {
            printf("Child process running...\n");
            sleep(2);
        }
    }

    /* Parent assigns child to its own group */
    setpgid(child, child);
    printf("Job started. PID = %d\n", child);
    printf("Press Ctrl+Z to suspend the job.\n");

    while (1) {
        waitpid(child, &status, WUNTRACED);
        if (WIFSTOPPED(status)) {
            job_status = 0;
            printf("Job stopped. Status saved.\n");
            printf("Press ENTER to resume...\n");
            getchar();
            getchar();
            /* Resume process group */
            kill(-child, SIGCONT);
            job_status = 1;
            printf("Job resumed.\n");
        }
    }
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano sigtstp.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc sigtstp.c -o sigtstp
shashank@LAPTOP-24KPROR8:~/OSLab$ ./sigtstp
Job started. PID = 1552
Child process running...
Press Ctrl+Z to suspend the job.
Child process running...
Child process running...
Child process running...
Child process running...
^Z
SIGTSTP received. Suspending job...
Job stopped. Status saved.
Press ENTER to resume...
```

19

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void LogError(const char *msg)
{
    FILE *fp = fopen("error.log", "a");
    if (fp != NULL)
    {
        fprintf(fp, "ERROR: %s\n", msg);
        fclose(fp);
    }
}

int executeCommand(char *cmd)
{
    if (strcmp(cmd, "start") == 0) {
        printf("Command executed successfully.\n");
        return 0;
    }
    if (strcmp(cmd, "stop") == 0) {
        printf("Process stopped successfully.\n");
        return 0;
    }
    if (strcmp(cmd, "status") == 0) {
        printf("System is running.\n");
        return 0;
    }
    printf("Error: Invalid command syntax.\n");
    LogError("Invalid command entered");
    return -1;
}

int main()
{
    char command[50];
    printf("Simple Error Handling System\n");
    while (1) {
        printf("\nEnter command (start/stop/status/exit): ");
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = 0;

        if (strcmp(command, "exit") == 0)
            break;

        if (strlen(command) == 0) {
            printf("Error: Empty command.\n");
            LogError("Empty command");
            continue;
        }
```

```
        if (executeCommand(command) != 0) {
            printf("Recovering... Please enter a valid command.\n");
        }
    }
    printf("Program terminated safely.\n");
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano controlerrors.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc controlerrors.c -o controlerrors
shashank@LAPTOP-24KPROR8:~/OSLab$ ./controlerrors
Simple Error Handling System

Enter command (start/stop/status/exit): start
Command executed successfully.

Enter command (start/stop/status/exit): stop
Process stopped successfully.

Enter command (start/stop/status/exit): status
System is running.

Enter command (start/stop/status/exit): exit
Program terminated safely.
```

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, sum = 0;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("ERROR: Invalid input\n");
        return 1;
    }

    int *arr = malloc(n * sizeof(int));
    if (arr == NULL) {
        printf("ERROR: Memory allocation failed\n");
        return 1;
    }

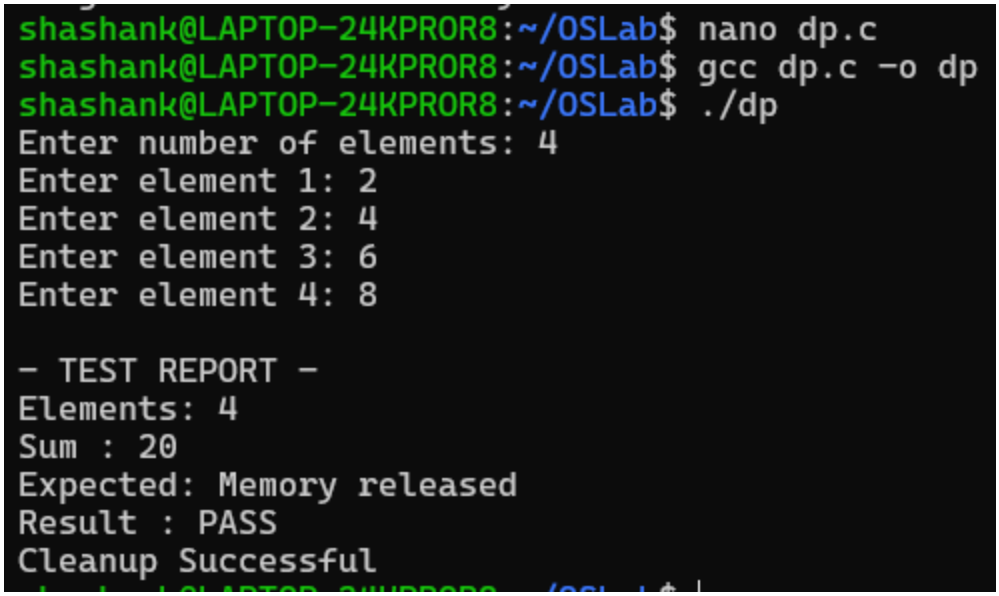
    for (int i = 0; i < n; i++) {
        printf("Enter element %d: ", i + 1);
        if (scanf("%d", &arr[i]) != 1) {
            printf("ERROR: Invalid value\n");
            free(arr);
            return 1;
        }
        sum += arr[i];
    }

    printf("\n- TEST REPORT -\n");
    printf("Elements: %d\n", n);
    printf("Sum : %d\n", sum);
    printf("Expected: Memory released\n");

    /* Release allocated memory */
    free(arr);
    arr = NULL;

    printf("Result : PASS\n");
    printf("Cleanup Successful\n");
    return 0;
}
```

Output:


shashank@LAPTOP-24KPROR8:~/OSLab$ nano dp.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc dp.c -o dp
shashank@LAPTOP-24KPROR8:~/OSLab$ ./dp
Enter number of elements: 4
Enter element 1: 2
Enter element 2: 4
Enter element 3: 6
Enter element 4: 8
- TEST REPORT -
Elements: 4
Sum : 20
Expected: Memory released
Result : PASS
Cleanup Successful

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int pipe1[2], pipe2[2];
    pid_t p1, p2, p3;
    int status;

    pipe(pipe1);
    pipe(pipe2);

    /* Process 1: ls */
    p1 = fork();
    if (p1 == 0) {
        dup2(pipe1[1], STDOUT_FILENO);
        close(pipe1[0]);
        close(pipe1[1]);
        execlp("ls", "ls", NULL);
        perror("ls failed");
        exit(1);
    }

    /* Process 2: grep */
    p2 = fork();
    if (p2 == 0) {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        execlp("grep", "grep", ".c", NULL);
        perror("grep failed");
        exit(1);
    }

    /* Process 3: wc */
    p3 = fork();
    if (p3 == 0) {
        dup2(pipe2[0], STDIN_FILENO);
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        execlp("wc", "wc", "-l", NULL);
        perror("wc failed");
        exit(1);
    }
```

```c
    /* Parent closes pipes */
    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    /* Monitor processes */
    waitpid(p1, &status, 0);
    printf("Job 1 completed.\n");
    waitpid(p2, &status, 0);
    printf("Job 2 completed.\n");
    waitpid(p3, &status, 0);
    printf("Job 3 completed.\n");

    printf("\nPipeline execution completed successfully.\n");
    return 0;
}
```

Output:

```
shashank@LAPTOP-24KPROR8:~/OSLab$ nano mp.c
shashank@LAPTOP-24KPROR8:~/OSLab$ gcc mp.c -o mp
shashank@LAPTOP-24KPROR8:~/OSLab$ ./mp
error: unexpected argument '-1' found

  tip: to pass '-1' as a value, use '-- -1'

Usage: wc [OPTION]... [FILE]...

For more information, try '--help'.
Job 1 completed.
Job 2 completed.
Job 3 completed.

Pipeline execution completed successfully.
```